Despliegue de Software OperPan

Aprendices

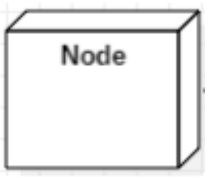
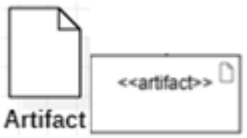
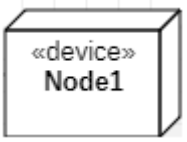
Eddier Antonio Paz Pardo
Santiago Muñeton Hernandez
Paula Andrea Herrera Carrillo

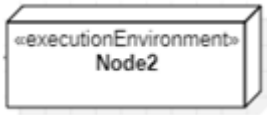
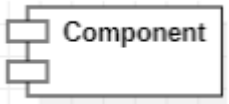
Instructor Angélica Triana

Servicio Nacional de Aprendizaje
Análisis y Desarrollo de Software
Ficha 3171608
20 de agosto de 2026

Actividad 1:

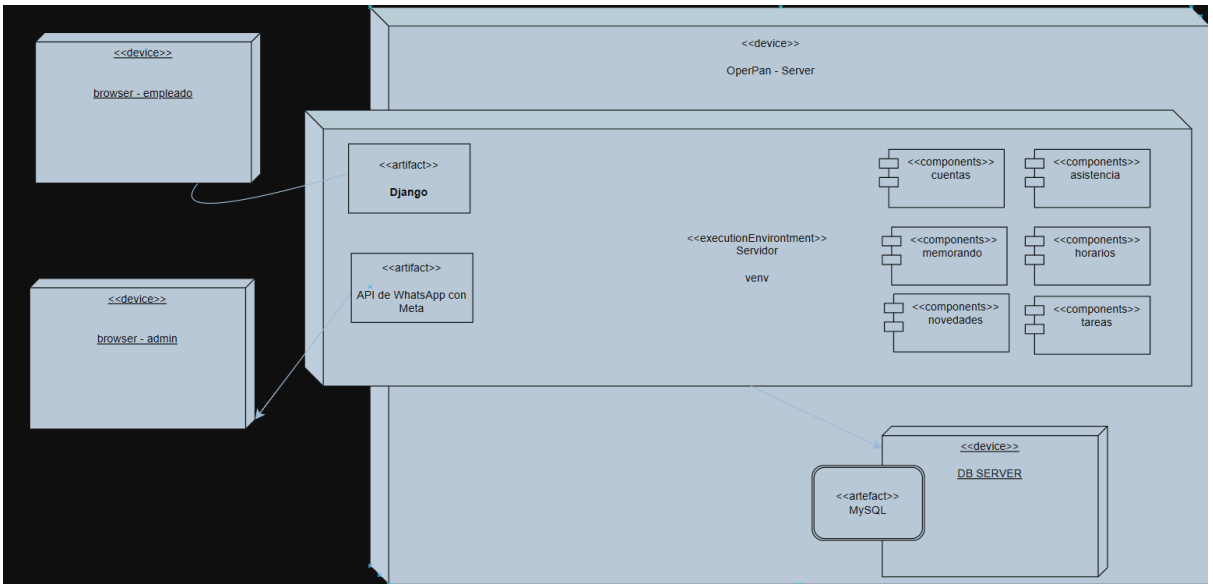
Esta actividad tiene como objetivo relacionar los elementos del diagrama de despliegue con los elementos que ha generado en su proyecto de software. Diligencie el siguiente cuadro:

Elemento	**Símbolo**	**Descripción**	¿Cómo estaría relacionado con mi proyecto?
Nodo		Representa un recurso físico o virtual (servidor, PC, móvil) donde se ejecuta el software.	El Servidor en la nube de Render (donde se ejecuta el backend), la PC del Administrador y el Navegador Web del Empleado.
Artefacto «artifact»		Representa un archivo o componente de software que se despliega en un nodo.	El archivo manage.py, el requirements.txt, los templates HTML (como landingAdmin.html) y los archivos PDF generados con ReportLab.
Comunicación (Asociación)		Línea que conecta nodos para indicar que se comunican.	La conexión segura HTTPS entre el navegador del usuario y el servidor Django en Render, y la conexión TCP de Render hacia la base de datos MySQL externa.
Dependencia		Una relación dirigida donde un elemento (el cliente) depende de la funcionalidad de otro (proveedor) para operar.	El Frontend (HTML/JS) depende del Backend (Django) para cargar los datos. A su vez, Django depende de librerías externas como ReportLab y PyMySQL para funcionar.
Estereotipo «device»		Es un subtipo de nodo que representa un dispositivo físico de hardware específico.	El PC del Administrador (para gestionar el sistema), el PC del Empleado y el Dispositivo Móvil (que usan los empleados para registrar su asistencia).

Estereotipo «executionEnvironment»		Un entorno de software que proporciona el contexto lógico necesario para ejecutar artefactos.	Render (nube donde corre Gunicorn y Django), MySQL (servidor de base de datos) y el Navegador Web (que ejecuta el JavaScript de Chart.js).
Componente		Una parte modular y reemplazable de un sistema que encapsula un conjunto de interfaces y comportamientos lógicos.	Las 6 aplicaciones lógicas de Django: usuarios, login, asistencia, tareas, novedades y memorandos.

Activada 2:

Diagrama



Activada 3:

En grupos de proyecto investigar y desarrollar un cuadro comparativo sobre al menos 3 posibles hostings que les permita desplegar su proyecto teniendo en cuenta: Costos, disponibilidad, bondades del site, uso de Cpanel, restricciones del site, características, paso a paso de instalación (a grandes rasgos). Compartir los hallazgos en la sesión.

Criterio	**Render**	**PythonAnywhere**	**Railway**
Costos	Gratis (con límites, se duerme a los 15 min) o **$7/mes** por servicio web. La base de datos PostgreSQL cuesta aparte (desde $6/mes) .	Gratis (limitado, 1 app y 100 seg CPU/día). Plan Developer cuesta $10/mes (1 app web, 5 GB almacenamiento) .	Gratis ($1 de crédito). Plan **Hobby $5/mes** (incluye $5 de consumo). Pagas por uso (10/GB) y CPU ($20/vCPU) .
Disponibilidad	Buena. El plan gratuito "duerme" tras 15 min de inactividad (tarda ~1 min en despertar). Los planes pagos son estables .	Excelente para Python. Al ser un entorno específico, es muy estable, pero los recursos son limitados .	Alta. Infraestructura moderna. El plan pago garantiza recursos dedicados.
Bondades del Site	Muy fácil de usar. Despliegue automático desde GitHub. Ideal para principiantes y equipos pequeños .	Entorno 100% Python. No tienes que configurar servidores web (Nginx/Apache) ni WSGI manualmente, ellos lo gestionan	Muy flexible. Panel sencillo, facturación por minuto. Buen soporte para bases de datos y workers en segundo plano .
Uso de cPanel	No usa cPanel. Es una plataforma administrada (PaaS). La gestión es vía Dashboard web y archivos de configuración.	No usa cPanel. La gestión es vía interfaz web propia (pestañas Web, Files, Consoles).	No usa cPanel. Gestión vía Dashboard y CLI (línea de comandos).
Restricciones	El plan Free no sirve para producción (se duerme y la BD gratis se borra a los 30 días). Requiere configurar WhiteNoise o similar para archivos estáticos .	El plan Free tiene CPU muy limitada (100 seg/día). El plan pago solo permite 1 app web (no podrías tener múltiples entornos fácilmente) .	El plan Free es solo para pruebas. El costo puede variar según el consumo (si la app usa mucha RAM, la factura sube) .

Características Técnicas	Soporta Python, Django, MySQL (vía externa), PostgreSQL nativo. Despliegue vía render.yaml o manual .	Soporta Python, Django, MySQL (en planes pagos). WSGI pregrabado. Ideal para apps tradicionales .	Soporta Python, Django, PostgreSQL, MySQL (vía plugin o externa). Detección automática de manage.py .
Paso a Paso (A grandes rasgos)	1. Subir código a GitHub. 2. Crear cuenta en Render y conectar repo. 3. Configurar Build Command (pip install -r requirements.txt) y Start Command (gunicorn config.wsgi). 4. Agregar variables de entorno (SECRET_KEY, DATABASE_URL). 5. Desplegar .	1. Subir código (vía Git o Files). 2. Crear Web App (Manual Config) seleccionando Python/Django. 3. Configurar Virtualenv e instalar requirements.txt. 4. Editar archivo WSGI para apuntar a config.settings. 5. Configurar rutas de archivos estáticos .	1. Subir código a GitHub. 2. En Railway, "New Project" -> "Deploy from GitHub". 3. Agregar base de datos (PostgreSQL o MySQL) como plugin. 4. Configurar variables de entorno (usando referencias como ${{MySQL.DATABASE_URL}}). 5. Railway detecta Django y despliega .